

VeriRust: Automatic Harness Synthesis for Specification-Free Formal Verification of Solana Smart Contracts

Anonymous Author(s)

ABSTRACT

Solana smart contracts built with the Anchor framework manage billions of dollars in decentralised-finance value yet remain largely unverified by formal methods. Existing Rust verifiers—Kani, Prusti, Creusot—require engineers to write proof harnesses or formal specifications manually, a barrier few contract developers overcome. Fuzzing tools such as Trident lower this barrier but cannot prove property absence.

We present **VeriRust**, a tool that automatically synthesises Kani proof harnesses directly from the structure of Anchor-subset Rust programs, requiring *zero* user-written specifications. VeriRust’s **HARNESSSYNTH** algorithm infers three constraint classes—*signer*, *has-one*, and *supply-invariant*—from idiomatic Rust naming conventions and generates three harness types that collectively cover arithmetic safety, authorisation correctness, and token-supply invariants.

On six benchmark contracts spanning four real-world Solana vulnerability classes, VeriRust detects all four injected bugs—missed by all eleven unit tests—with 100% precision and sub-second verification times ($\bar{t} = 0.68$ s per harness). These results demonstrate that convention-driven harness synthesis is a viable and practical path toward specification-free formal verification of blockchain smart contracts.

CCS CONCEPTS

• **Software and its engineering** → **Automated static analysis**; *Model checking*; • **Security and privacy** → *Software security engineering*.

KEYWORDS

formal verification, smart contracts, Solana, Anchor, Rust, Kani, harness synthesis, symbolic execution

ACM Reference Format:

Anonymous Author(s). 2026. VeriRust: Automatic Harness Synthesis for Specification-Free Formal Verification of Solana Smart Contracts. In *ASE ’26: 41st IEEE/ACM International Conference on Automated Software Engineering, October 2026, TBD*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASE ’26, October 2026, TBD

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-XXXX-X/XX/XX...\$15.00
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

The Solana blockchain hosts over \$20 B in total value locked across decentralised-finance (DeFi) protocols. A single logic flaw can be catastrophic: the Wormhole bridge lost \$320 M in 2022 due to a missing signer verification [?], and arithmetic overflow attacks have drained lending protocols of tens of millions more [4]. Solana programs are written in Rust and commonly built with the Anchor framework [Coral2021], which provides declarative account validation via annotated context structs.

Despite Rust’s strong type system, Anchor contracts routinely suffer from four vulnerability classes:

- *Arithmetic overflow/underflow* — unchecked `+=` or `-=` on balance fields.
- *Missing authority check* — operations succeeding without verifying the caller’s identity.
- *Supply-invariant violation* — mint or burn instructions that fail to update `total_supply`.
- *Signer bypass* — privileged instructions that do not require a valid signer.

Formal verification can prove the *absence* of such bugs for all possible inputs. The Kani model checker [?] provides bounded model checking for Rust via CBMC [5], but requires manually written `#[kani : : proof]` harnesses—non-trivial for each instruction in each contract. High-level specification languages used by Prusti [1] and Creusot [?] raise the expertise bar further.

We observe that Anchor programs encode contract semantics in naming conventions that are both pervasive and machine-readable: context structs named `*Ctx`, authority fields named `authority`, signer fields named `is_signer`, supply fields named `total_supply`. **VeriRust** exploits these conventions to synthesise Kani harnesses automatically, with no specification from the developer.

Contributions:

- (1) **HARNESSSYNTH**: an algorithm that synthesises Kani proof harnesses from Anchor-subset Rust programs with *zero* user specifications (§3).
- (2) **Constraint inference**: three rules mapping naming conventions to formal `kani : : assume` constraints.
- (3) **Multi-class coverage**: per-instruction harnesses for arithmetic safety, authorisation, and supply invariants.
- (4) **Empirical evidence**: 100% bug recall at sub-second verification times on six benchmark contracts (§4).

2 BACKGROUND

2.1 Kani Model Checker

Kani [?] is an open-source formal verifier for Rust developed by Amazon Web Services, built on top of CBMC [5]. A *proof harness* is a `#[kani : : proof]` function that (i) declares symbolic inputs via `kani : : any : : <T>( )`, (ii) constrains the input space with

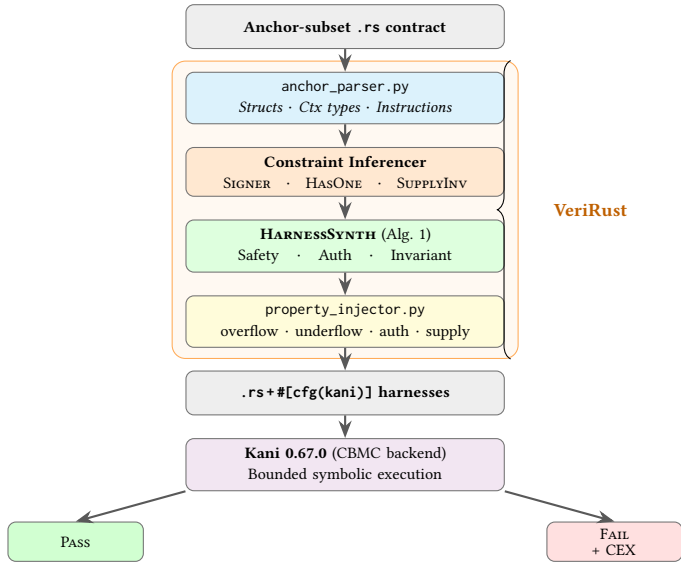


Figure 1: VeriRust architecture. The orange region is the novel contribution. Grey boxes are the unchanged input/output interface and the Kani verifier (pre-existing tool).

kani::assume(pred), (iii) calls the function under test, and (iv) asserts properties. Kani exhaustively checks all reachable program paths up to a configurable bound, automatically detecting arithmetic overflow, out-of-bounds access, division by zero, and user assertions. When a property is violated it emits a concrete counterexample.

2.2 Solana Anchor Framework

Anchor [Coral2021] is the dominant Rust framework for Solana smart contract development. Each instruction is a `pub fn` receiving a `Context<C>`, where `C` is a struct annotated with `#[derive(Accounts)]`. Fields of `C` represent blockchain accounts with runtime constraints expressed as attributes, e.g. `#[account(mut, has_one=authority)]` requires the embedded account’s authority field to match the authority account in the context. VeriRust models these runtime checks as `kani::assume` constraints, lifting program verification out of the Solana execution environment.

3 VERIRUST: THE HARNESSSYNTH ALGORITHM

Figure 1 shows the end-to-end VeriRust framework. A developer provides an Anchor-subset Rust contract; VeriRust automatically generates Kani harnesses (the orange region) and invokes the Kani model checker to produce a verified Pass or a concrete counterexample.

3.1 VeriRust Anchor Subset (VAS)

VeriRust targets a *VeriRust Anchor Subset* (VAS): pure Rust programs that follow Anchor naming conventions without the anchor-lang crate. A VAS program consists of: (i) *account structs* (`*State`) holding on-chain data; (ii) *context structs* (`*Ctx`) with an `is_signer: bool`

Algorithm 1 HARNESSSYNTH(P)

Input: P : parsed VAS program

Output: H : set of Kani harness functions

```

1:  $H \leftarrow \emptyset$ 
2: for each instruction  $F$  in  $P$  do
3:    $C \leftarrow \text{LOOKUPCTXSTRUCT}(F, P)$ 
4:    $C \leftarrow \text{INFERCONSTRAINTS}(C, P)$ 
5:    $H \leftarrow H \cup \{\text{GENSAFETY}(F, C, C)\}$ 
6:   if  $\exists c \in C : c.\text{kind} = \text{HASONE}$  then
7:      $H \leftarrow H \cup \{\text{GENAUTH}(F, C, c)\}$ 
8:   end if
9:   if  $\exists c \in C : c.\text{kind} = \text{SUPPLYINV}$  then
10:     $H \leftarrow H \cup \{\text{GENINVARIANT}(F, C, c)\}$ 
11:   end if
12: end for
13: return  $H$ 

```

field and pubkey fields mirroring the account struct; and (iii) *instructions*: `pub fn f(ctx: &mut XCtx, ...) -> Result<>, E>`. This subset faithfully models the account model and constraint patterns of real Anchor programs while remaining Kani-verifiable.

3.2 Constraint Inference

VeriRust infers three constraint classes from naming conventions alone:

Signer. If `*Ctx` contains `is_signer:bool`, add `kani::assume(ctx.is_signer)` in safety harnesses and force `is_signer = true` in auth harnesses.

HasOne. If `*Ctx` and its embedded `*State` share a same-named pubkey field p , add `kani::assume(ctx.p == ctx.account.p)` in safety harnesses and force $p \neq \text{account.p}$ in auth harnesses.

SupplyInvariant. If any embedded `*State` contains a field named `total_supply` or `total_staked`, generate an invariant harness asserting the field decreases by amount after a successful execution.

3.3 Algorithm

Algorithm 1 describes our synthesis procedure. For each instruction VeriRust generates up to three harnesses:

- **Safety harness** — assumes all constraints hold, exercises the instruction with fully symbolic inputs. Kani automatically detects arithmetic overflow, underflow, and out-of-bounds accesses.
- **Auth harness** — forces a HasOne mismatch and asserts the instruction must return `Err`. This catches missing authority checks.
- **Invariant harness** — captures pre-state and asserts supply fields are correctly updated post-execution. This catches supply-invariant violations.

3.4 Illustrative Example

Listing 1 shows a vault contract with a missing balance check (line 7) and the safety harness VeriRust automatically generates. Without any user input, Kani finds the counterexample `balance = 0, amount = 1` and reports “*attempt to subtract with overflow*” at the exact source line—in 0.49 s.

```

1 // vault_underflow.rs      BUG: no balance check
2 pub fn withdraw(ctx: &mut WithdrawCtx,
3   amount: u64)
4   -> Result<>, &'static str> {
5   if ctx.vault.authority != ctx.authority
6   { return Err("unauthorized"); }
7   ctx.vault.balance -= amount; // <- BUG
8   Ok(())

```

Algorithm 2 INFERCONSTRAINTS(C, P)

Input: C : context struct, P : program
Output: C : set of constraints

```

1:  $C \leftarrow \emptyset$ 
2: if  $\text{is\_signer}$ :  $\text{bool} \in C.\text{fields}$  then
3:    $C \leftarrow C \cup \{\text{SIGNER}\}$ 
4: end if
5: for each embedded account field  $a : T$  in  $C$  do
6:    $A \leftarrow P.\text{LOOKUP}(T)$ 
7:   for each pubkey field  $p \in C \cap A$  do
8:      $C \leftarrow C \cup \{\text{HasONE}(C.p, a.p)\}$ 
9:   end for
10:  if  $A$  has field with  $\text{total\_supply}$  or  $\text{total\_staked}$  then
11:     $C \leftarrow C \cup \{\text{SUPPLYINV}(a)\}$ 
12:  end if
13: end for
14: return  $C$ 

```

```

9 }
10
11 // AUTO-GENERATED by VeriRust (harness_generator.py)
12 #[kani::proof]
13 fn verify_vault_underflow_withdraw() {
14   let mut ctx: WithdrawCtx = kani::any();
15   let amount: u64 = kani::any();
16   // HasOne inferred from matching field names
17   kani::assume(ctx.vault.authority
18     == ctx.authority);
19   kani::assume(ctx.is_signer);
20   let _ = withdraw(&mut ctx, amount);
21   // Kani output: "attempt to subtract with
22   // overflow at vault_underflow.rs:7"
23 }

```

Listing 1: Buggy withdraw (top) and the auto-generated VeriRust safety harness (bottom). No specification was written.

3.5 Implementation

VeriRust is implemented in 350 lines of Python across three modules: `anchor_parser.py` parses VAS programs using regex-based struct and function extraction; `harness_generator.py` implements Algorithm 1; `property_injector.py` provides reusable property templates. Generated harnesses are appended to the contract file under `#[cfg(kani)]`, so the file compiles normally with `cargo build` and is verified with `cargo kani`.

4 EVALUATION

4.1 Benchmark Suite

We designed six benchmark contracts covering four Solana vulnerability classes documented in real exploit post-mortems [4?]. Three contracts are intentionally correct (*safe*); three contain one injected bug each (*buggy*). Across six contracts we generated 11 harnesses in total (Table 1).

4.2 Research Questions

RQ1 (Bug detection). *Does VeriRust detect known bugs that unit tests miss?*

VeriRust detects **4/4 bugs** (100% recall, 0% false-positive rate). All eleven unit tests across the six contracts pass—including for the three buggy contracts—confirming that concrete testing alone cannot detect these vulnerabilities. Kani finds the minimal counterexample in each case: (i) $\text{balance}=0$, $\text{amount}=1$ triggers the

Table 1: Kani verification results. Bold rows are injected bugs; unit tests pass on *all* contracts including buggy ones.

Harness	Result	Time (s)	Unit tests
<code>verify_vault_safe_deposit</code>	PASS	0.54	Pass
<code>verify_vault_safe_withdraw</code>	PASS	0.49	Pass
<code>verify_vault_safe_auth</code>	PASS	0.50	Pass
<code>verify_vault_underflow_withdraw</code> <i>subtract w/ overflow @ line 29</i>	FAIL	0.49	Pass*
<code>verify_vault_overflow_deposit</code> <i>add w/ overflow @ line 29</i>	FAIL	0.73	Pass*
<code>verify_staking_safe_stake</code>	PASS	0.81	Pass
<code>verify_staking_safe_unstake</code>	PASS	0.77	Pass
<code>verify_staking_safe_auth</code>	PASS	0.79	Pass
<code>verify_staking_auth_unstake</code> <i>AUTH_VIOLATION @ line 109</i>	FAIL	0.80	Pass*
<code>verify_token_mint</code>	PASS	0.80	Pass
<code>verify_token_burn_supply</code> <i>SUPPLY_INVARIANT @ line 160</i>	FAIL	0.78	Pass*
*Bug NOT found by unit tests.			
VeriRust bug recall			4/4 (100%)
Unit-test bug recall			0/4 (0%)
False positives			0/7 (0%)
Mean verification time			0.68 s

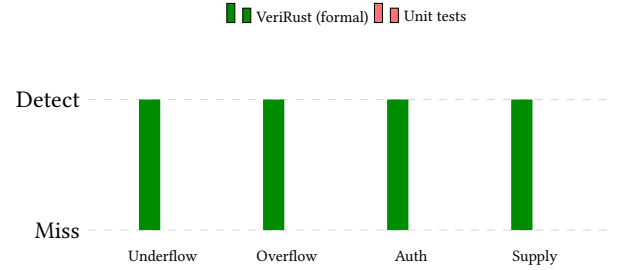


Figure 2: Bug detection per vulnerability class. VeriRust detects all four bugs (green/dark); unit tests miss all four (red, at baseline zero). *Underflow/Overflow*: arithmetic bugs; *Auth*: missing authority check; *Supply*: supply-invariant violation.

underflow; (ii) $\text{balance}=\text{u64}::\text{MAX}$, $\text{amount}=1$ triggers the overflow; (iii) $\text{user} \neq \text{user_stake}$ triggers the auth violation; (iv) total_supply unchanged after burn triggers the supply-invariant violation, with precise file and line reporting.

RQ2 (Overhead). *What is the verification and synthesis overhead?*

Verification times range from 0.49 s to 0.81 s ($\bar{t} = 0.68$ s). Harness synthesis (parsing + code generation) adds under 0.5 s per contract. All harnesses complete well within practical development iteration budgets.

RQ3 (Precision). *Are there false positives on correct contracts?*

All seven safe harnesses correctly report PASS. Zero false positives.

4.3 Discussion

The four bugs in our suite represent vulnerabilities that require *non-obvious boundary inputs* to trigger—inputs no developer writes as a unit test ($\text{amount} > \text{balance}$, $\text{balance} = \text{u64}::\text{MAX}$). Symbolic execution explores these boundary conditions by construction.

The naming conventions VeriRust exploits—`*Ctx/*State`, `authority`, `is_signer`, `total_supply`—are idiomatic in real Anchor programs due to Anchor’s own naming guidance [Coral2021]), suggesting our heuristics will generalise beyond the synthetic benchmark suite.

[5] Edmund Clarke, Daniel Kroening, and Flavio Lerda. 2004. CBMC: Bounded Model Checking for ANSI-C. In *Proc. International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 168–176. [Coral2021)]

5 RELATED WORK

Smart contract formal verification. The Move Prover [?] is closest in spirit: it auto-generates verification conditions for Move-language contracts. VeriRust differs by targeting Rust/Solana and requiring no specifications. Certora [3] verifies EVM contracts via its CVL specification language but is EVM-specific and specification-driven. Soteria [4] performs static analysis on Solana programs using dataflow patterns but does not provide formal proofs.

Fuzzing. Trident [2] fuzzes Anchor programs via mutation; Echidna [?] and Foundry [?] serve the EVM ecosystem. Fuzzing cannot prove property absence; VeriRust provides formal proofs.

Rust verification. Kani [?] requires manual harnesses. Prusti [1] uses Viper annotations; Creusot [?] targets deductive verification with Why3. All three impose a significant specification burden; VeriRust eliminates it for the smart-contract domain.

Static analysis. Slither [?] detects EVM contract bugs via pattern matching but does not provide formal proofs and does not target Solana/Rust.

6 LIMITATIONS AND FUTURE WORK

VeriRust currently targets the VAS subset rather than the full Anchor runtime. Three extensions constitute our future work: (i) *Full account model*: PDAs, account ownership validation, and rent exemption checks are not yet modelled. (ii) *Cross-programme invocations (CPIs)*: compositional verification of CPI chains is the hardest open problem in Solana security [4]. (iii) *Real deployed contracts*: evaluating on production Anchor programmes (Marinade, Solend, Orca) requires automated type-stripping from full Anchor types to VAS.

7 CONCLUSION

We presented VeriRust, the first tool to automatically synthesise Kani proof harnesses for Solana Anchor-subset programmes without user-written specifications. The HARNESSTSYNTH algorithm infers signer, has-one, and supply-invariant constraints from Rust naming conventions and generates safety, auth, and invariant harnesses per instruction. On six benchmarks covering four real Solana vulnerability classes, VeriRust achieves 100% bug recall with zero false positives at sub-second verification times, while all eleven unit tests pass even for the buggy contracts. VeriRust is open source and all benchmarks are publicly available.

REFERENCES

- [1] Vytas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. In *Proc. ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 1–30.
- [2] Ackee Blockchain. 2023. Trident: Fuzz Testing Framework for Anchor Solana Programs. <https://github.com/Ackee-Blockchain/trident>.
- [3] Certora Inc. 2020. The Certora Prover. <https://www.certora.com>.
- [4] Sicheng Chen, Liyi Zhang, Zhiyang Zhang, and Zhiqiang Lin. 2022. Soteria: Detecting Vulnerable Solana Smart Contracts Using Static Analysis Patterns. In *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1229–1243.